INFORME DE PRÁCTICA DE LABORATORIO — SESIÓN 6

Comunicación Síncrona Resiliente entre Servicios (OpenFeign y Circuit Breaker)

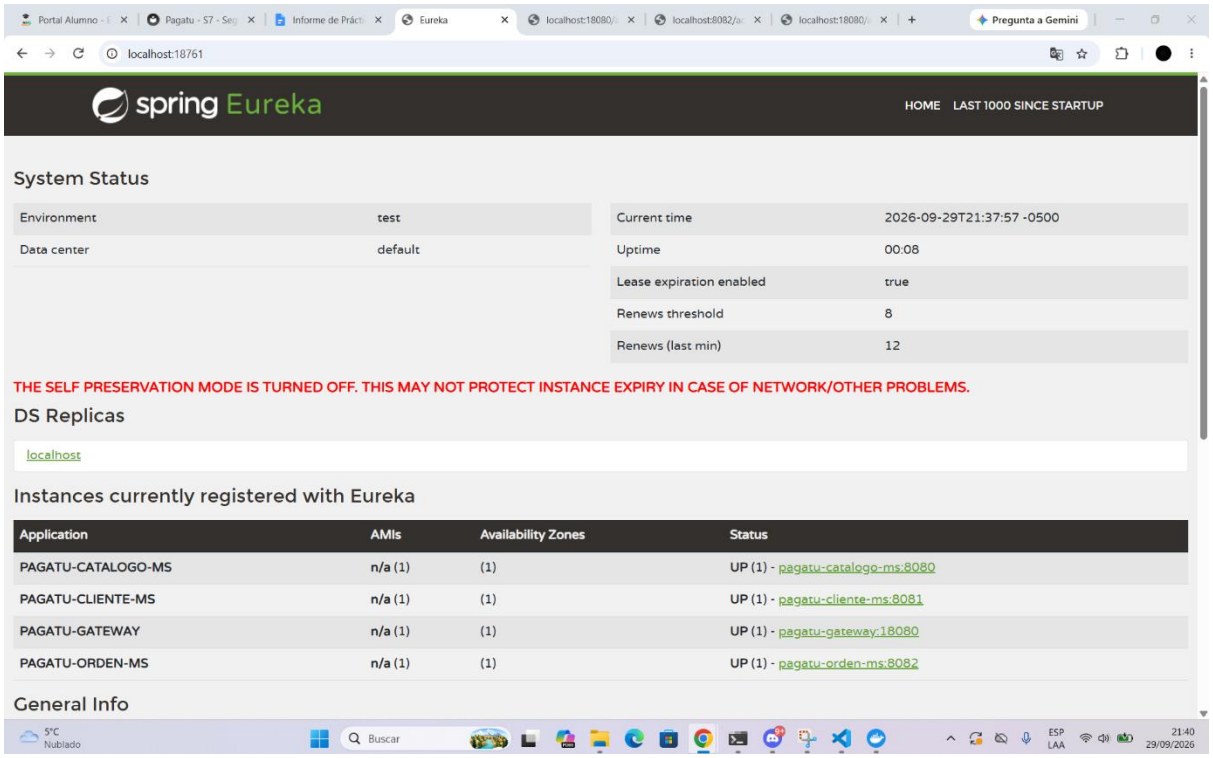
1. DATOS DEL ESTUDIANTE

- Nombre y Apellidos: Witmer Jhetly Mamani Cahuana
- Equipo de Trabajo: Equipo 01
- Sesión: S06 — Comunicación síncrona resiliente entre servicios
- Rol o Aporte Realizado: Implementación del microservicio pagatu-orden-ms integrando OpenFeign contra pagatu-catalogo-ms, configuración de tolerancia a fallos con Resilience4j (desacoplamiento de bean para evitar auto-invocación del proxy AOP), construcción autónoma del microservicio pagatu-cliente-ms con base de datos independiente y migración Flyway, registro en Eureka y configuración centralizada en Config Server.
- Enlace del Repositorio en GitHub: https://github.com/Witmer17/Feign-y-CB.git

2. EVIDENCIA TÉCNICA Y RESULTADOS

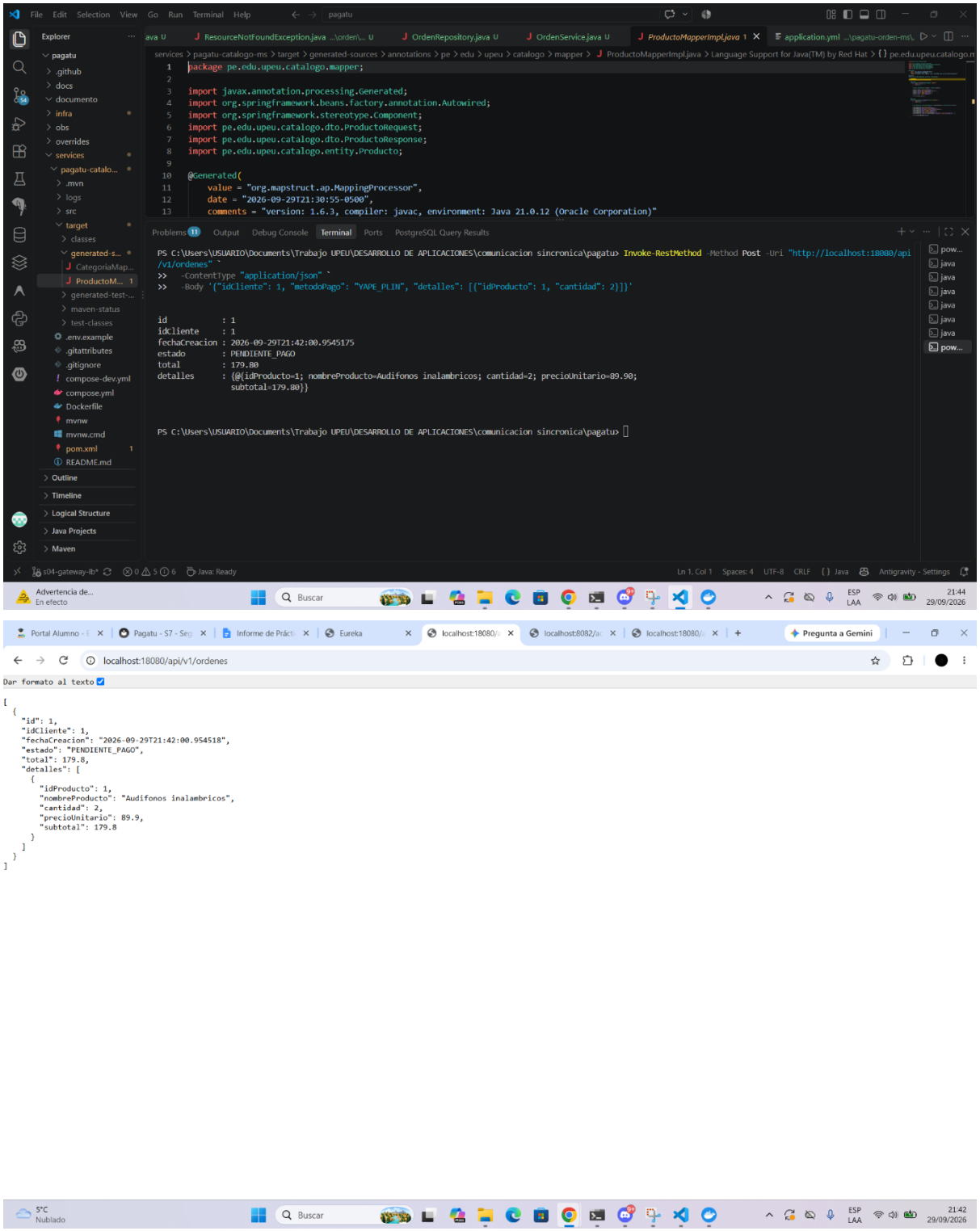
Dimensión 1: pagatu-orden-ms Construido y Registrado

El microservicio pagatu-orden-ms fue implementado en el puerto DEV 8082, respaldado por su propio contenedor Docker PostgreSQL (pagatu-postgres-orden-dev, puerto host 15434, base de datos pagatu_orden_db). Al arrancar, el microservicio carga su configuración externa de manera centralizada desde pagatu-config (http://localhost:18888/pagatu-orden-ms/dev) y se registra automáticamente como instancia activa en pagatu-eureka (http://localhost:18761).



Dimensión 2: Comunicación Declarativa con OpenFeign (Caso Exitoso / CLOSED)

Al recibir una solicitud POST `/api/v1/ordenes` a través de `pagatu-gateway` (puerto 18080), `pagatu-orden-ms` emplea el cliente declarativo `@FeignClient(name = "pagatu-catalogo-ms")`. Este cliente resuelve la ubicación de red dinámicamente mediante Eureka sin acoplar direcciones IP ni puertos fijos. Se consulta el endpoint `/api/v1/productos/{id}`, copiando el nombre y el precio real a la orden para garantizar la inmutabilidad histórica del documento comercial. La orden pasa inmediatamente a estado `PENDIENTE_PAGO` con el total aritméticamente consolidado.

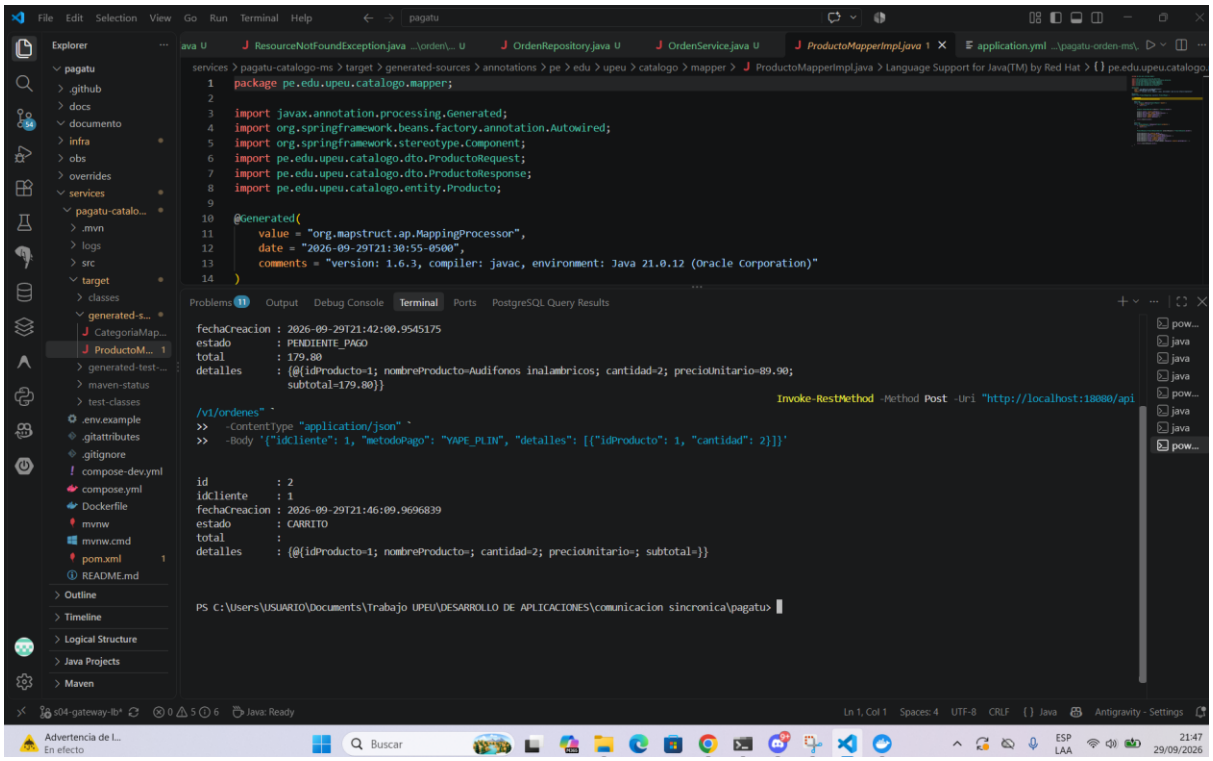


Dimensión 3: Tolerancia a Fallos con Circuit Breaker (Resilience4j)

Degradación Grácil ante Caída del Catálogo:

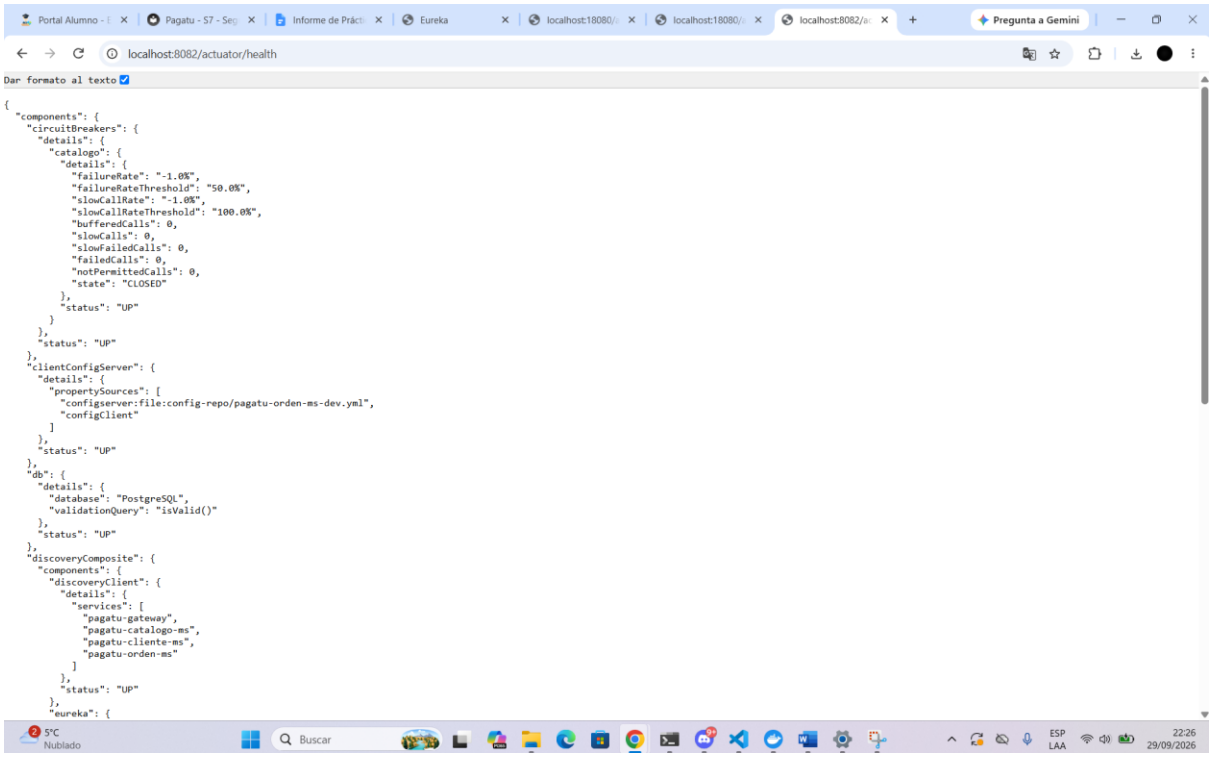
Al detener manualmente el proceso de `pagatu-catalogo-ms` y solicitar la creación de una orden, la llamada no propaga un error HTTP 500 al cliente. El método de fallback en `ProductoConsultaService` intercepta la excepción de conexión y

retorna `null`, permitiendo a la lógica de negocio persistir la orden en estado `CARRITO`, con total: `null` y precios en `null` para su posterior regularización.



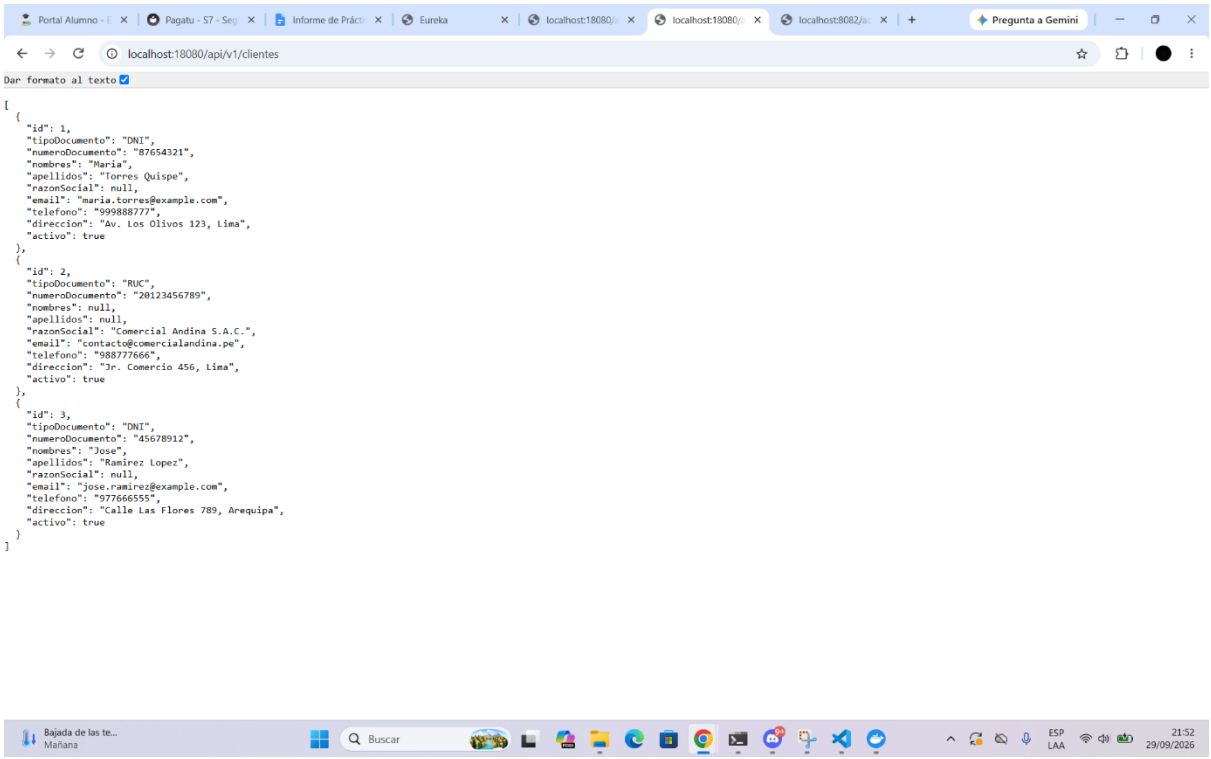
Transición y Evidencia del Estado OPEN:

Se enviaron más de 5 peticiones consecutivas fallidas con el fin de saturar la ventana deslizante por conteo (`sliding-window-size: 5`) y superar el umbral de fallas (`failure-rate-threshold: 50%`). Al consultar el endpoint Actuator `/actuator/health` en el puerto 8082, se comprueba que el circuito cambió a estado `OPEN`, cortando las llamadas de red y ejecutando el fallback en 0 ms.



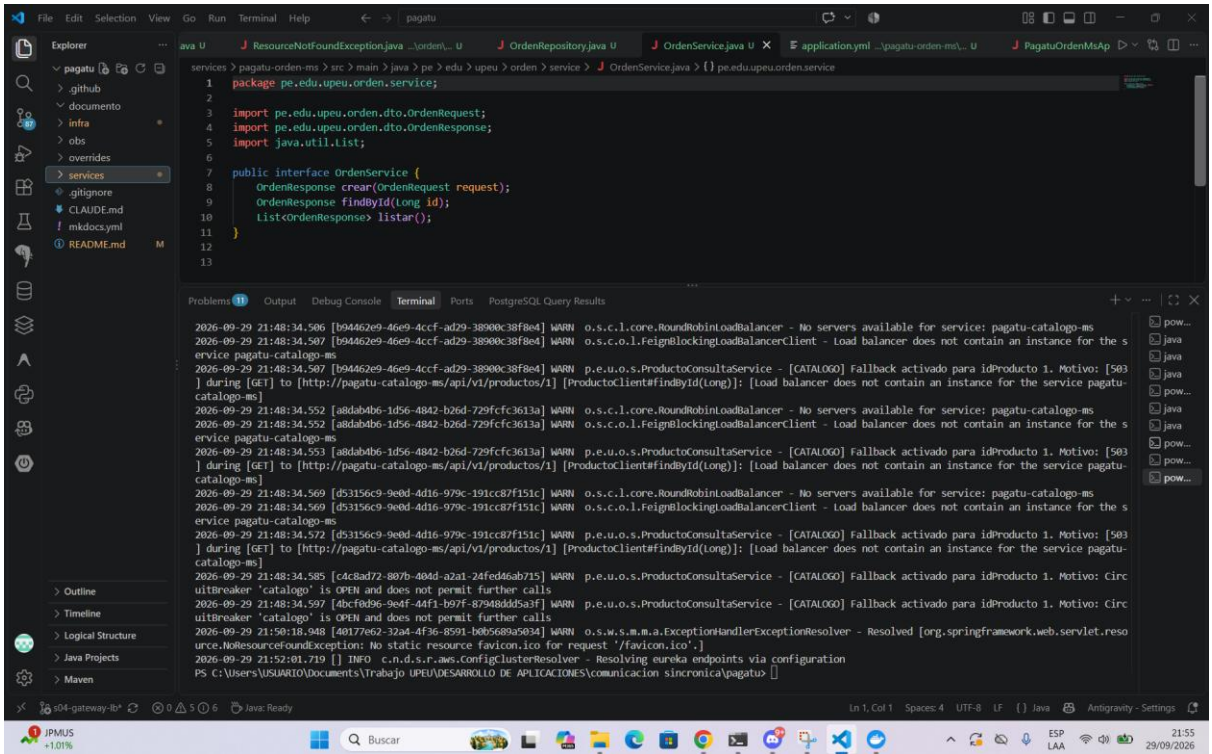
Dimensión 4: Microservicio pagatu-cliente-ms Construido (Actividad Autónoma)

Se implementó de punta a punta el microservicio `pagatu-cliente-ms` en el puerto 8081 bajo el paquete `pe.edu.upeu.cliente`. El servicio opera con un contenedor PostgreSQL independiente (`pagatu-postgres-cliente-dev`, puerto host 15433, base `pagatu_cliente_db`), migración versionada con Flyway (`V1__create_cliente_tables.sql`), repositorio JPA con búsqueda por documento, capa de servicio con CRUD completo, controlador en `/api/v1/clientes` y ruta habilitada en el API Gateway (`lb://pagatu-cliente-ms`).



Dimensión 5: Observabilidad y Diagnóstico de Trazabilidad

Mediante `CorrelationIdFilter`, cada petición entrante genera o propaga una cabecera `X-Trace-ID` inyectada en el MDC de SLF4J bajo la clave `traceId`. En el archivo `logs/orden.log`, se observa la trazabilidad correlacionada durante la falla: recepción de solicitud, advertencia del fallback interceptando la excepción y respuesta final con el mismo identificador.



3. ERROR O HALLAZGO TÉCNICO

- **¿Qué pasó en la práctica?:** Al principio puse la anotación @CircuitBreaker y el método `fallbackProducto()` dentro del mismo `OrdenServiceImpl` para ahorrar código y no crear más clases. Pensé que con eso bastaba, pero cuando detuve `pagatu-catalogo-ms` para probar el fallo, el servicio no activaba el fallback y me seguía reventando la consola con un error 500 de Feign (`FeignException$ServiceUnavailable`).
- ¿Por qué fallaba?: Investigando descubrí el problema de la auto-invocación (self-invocation) con los proxies de Spring. Como la llamada a this.consultarProducto() se hacía internamente dentro de la misma clase, la ejecución nunca pasaba por el proxy interceptor de Resilience4j. Para Java era solo un método local y por eso ignoraba el Circuit Breaker por completo.
- ¿Cómo lo solucioné?: Tuve que crear una clase aparte (ProductoConsultaService), mover ahí la consulta Feign con el @CircuitBreaker e inyectar ese nuevo servicio dentro de OrdenServiceImpl. Al llamar a otro bean distinto, Spring ya pudo interceptar la llamada y el fallback por fin funcionó guardando la orden en CARRITO.

4. JUSTIFICACIÓN ARQUITECTÓNICA: BASE DE DATOS INDEPENDIENTE

(Requisito de la Actividad Autónoma 4.1)

No se comparte una base de datos entre `pagatu-catalogo-ms`, `pagatu-orden-ms` y `pagatu-cliente-ms` porque violaría el principio fundamental de Database-per-Service. Compartir un esquema relacional crearía un acoplamiento estrecho entre esquemas, bloqueando despliegues independientes: un cambio en las columnas de una tabla podría romper los otros microservicios sin previo aviso. Además, impide escalar las bases de datos según sus necesidades específicas (las órdenes requieren transaccionalidad concurrente, mientras el catálogo prioriza lecturas masivas) y elimina el principio de autonomía de dominio, donde cada microservicio es el único dueño y guardián de sus datos.

5. REFLEXIÓN TÉCNICA

(Respuesta en 5 a 8 líneas al requerimiento de la sección 4.3.1)

Aumentar el timeout solo hace que la aplicación se quede esperando más segundos colgada antes de avisar que falló. Si el catálogo se cae y entran varias órdenes seguidas, todos los hilos del servidor se quedan congelados esperando respuesta y terminan tumbando también a `pagatu-orden-ms`. En cambio, el Circuit Breaker detecta cuando el servicio destino ya falló varias veces seguidas y directamente "abre el circuito". Al abrirse ya ni intenta conectarse por red: responde de inmediato en 0 milisegundos con el fallback, protegiendo la memoria y evitando que la caída de un microservicio afecte a los demás.

6. ANEXO: FEEDBACK DE LA SESIÓN

- **¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?**

Aprender que las anotaciones de Spring basadas en proxies (como Circuit Breaker o Transactional) no funcionan si te llamas a ti mismo dentro de la misma clase, y entender cómo degradar una respuesta a CARRITO en vez de mandarle un pantallazo de error 500 al usuario.

- **¿Qué punto de la clase te resultó más confuso o te dejó con dudas?**

Al principio me costó entender cómo calcula la ventana deslizante por conteo (Count-based) y por qué se necesitan exactamente 5 llamadas antes de que el estado pase a OPEN.

- **¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?**

¿Cómo se puede hacer para que el X-Trace-ID pase automáticamente en las cabeceras de Feign al siguiente servicio sin tener que copiarlo a mano en cada llamada?

- **Sobre tu nivel de comprensión de la clase de hoy, marca una opción:**
  [ ] ¡Entendido! — Lo domino y podría explicarlo.
  [X] Más o menos. — Entendí la idea general, pero tengo dudas.
  [ ] Necesito ayuda. — Me siento perdido/a con este tema.

- **¿Cómo puedo ayudarte a comprender mejor el tema?**

Haciendo un ejemplo más visual o viendo cómo cambia el estado del circuito en tiempo real en los paneles de Grafana.

- **Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?**
  [X] Muy Comprometido/a: Me esforcé al máximo.
  [ ] Comprometido/a: Sé que podría haberme esforzado un poco más.
  [ ] Poco Comprometido/a: Hoy no di mi mejor esfuerzo.
- **Mi satisfacción con la clase fue (del 1 al 10):**

Mi satisfacción fue de un 9